

PRACTICAL – 11

Name: S Rikhil

Roll No: 2520030480

Sec: 8

PART 1: MULTITHREADED COUNTER WITH RACE CONDITION

Aim

To develop a multithreaded counter application using POSIX threads (pthread_create() and pthread_join()) and demonstrate race conditions when multiple threads update a shared variable concurrently.

Theory

A thread is an independent execution unit within a process. POSIX threads provide functions such as pthread_create() to create threads and pthread_join() to wait for their completion.

When multiple threads access and modify the same shared variable simultaneously without synchronization, a **race condition** can occur. The final value may be different from the expected value.

Algorithm

1. Start the program.
2. Create a shared counter.
3. Create multiple threads using pthread_create().
4. Allow each thread to increment the shared counter.
5. Wait for all threads using pthread_join().
6. Display the final counter value.
7. Observe the race condition.
8. Stop.

C Program – Race Condition

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
int counter = 0;
```

```
void *increment(void *arg)
```

```
{  
    for (int i = 0; i < 100000; i++)  
    {  
        counter++;  
    }  
}
```

```
return NULL;
```

```

}

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final Counter Value: %d\n", counter);

    return 0;
}

```

Compilation

```
gcc race.c -o race -pthread
```

Execution

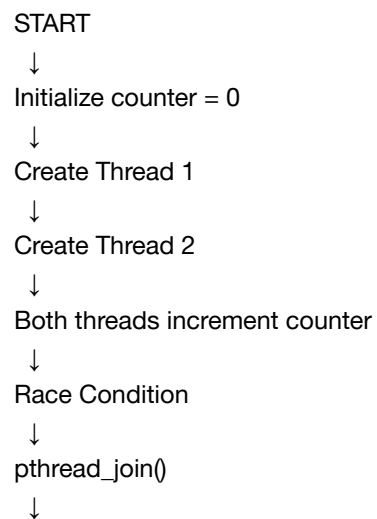
```
./race
```

Expected Output

Final Counter Value: 200000

However, due to the race condition, the output may sometimes be less than 200000.

Flow Chart



Display Counter

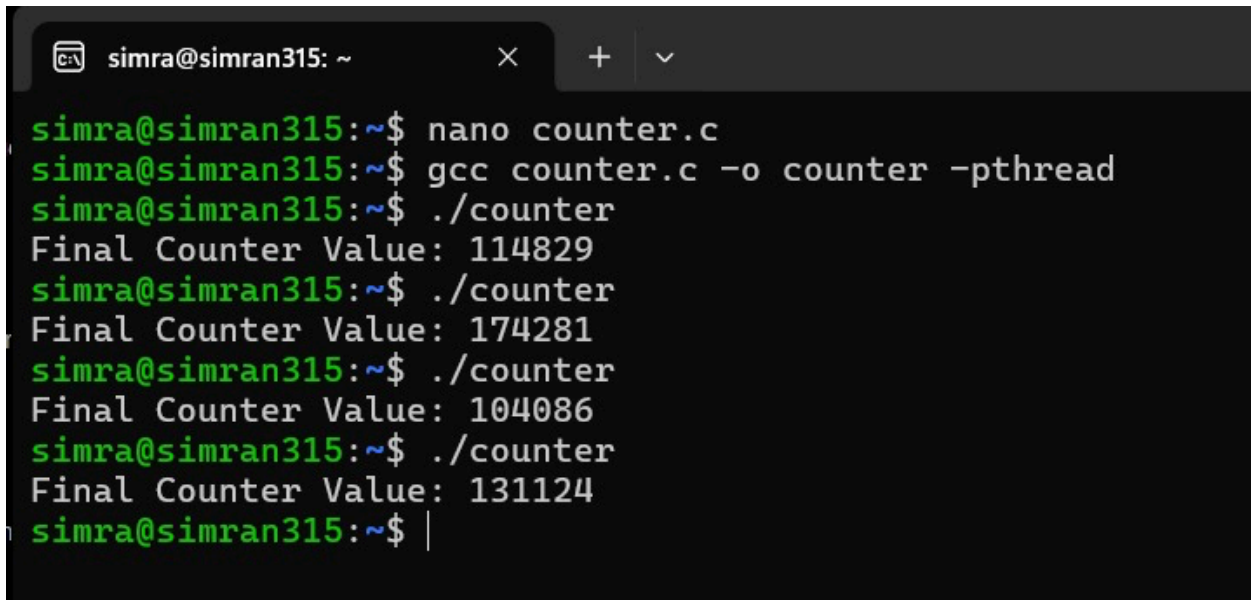


STOP

Observation

Both threads access the shared counter simultaneously without synchronization. Therefore, some increments may be lost due to the race condition.

Output:

A terminal window titled 'simra@simran315: ~' showing the execution of a C program. The user runs 'nano counter.c', 'gcc counter.c -o counter -pthread', and then runs the program multiple times. The output shows 'Final Counter Value' followed by different numbers: 114829, 174281, 104086, and 131124. The terminal text is as follows:

```
simra@simran315:~$ nano counter.c
simra@simran315:~$ gcc counter.c -o counter -pthread
simra@simran315:~$ ./counter
Final Counter Value: 114829
simra@simran315:~$ ./counter
Final Counter Value: 174281
simra@simran315:~$ ./counter
Final Counter Value: 104086
simra@simran315:~$ ./counter
Final Counter Value: 131124
simra@simran315:~$ |
```

Result

The multithreaded counter program was successfully implemented, and the race condition caused by concurrent access to the shared variable was demonstrated.

PART 2: COUNTER USING MUTEX LOCK

Aim

To modify the multithreaded counter program using `pthread_mutex_lock()` and `pthread_mutex_unlock()` and compare the output with the race-condition version.

Theory

A **mutex (mutual exclusion)** allows only one thread at a time to access a critical section.

```
pthread_mutex_lock(&mutex);
```

```
counter++;
```

```
pthread_mutex_unlock(&mutex);
```

The mutex prevents multiple threads from modifying the shared variable at the same time.

Algorithm

1. Start the program.
2. Initialize the shared counter.
3. Initialize the mutex.
4. Create multiple threads.
5. Lock the mutex before modifying the counter.
6. Increment the counter.
7. Unlock the mutex.
8. Wait for all threads using `pthread_join()`.
9. Display the final counter value.
10. Destroy the mutex.
11. Stop.

C Program – Mutex Version

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
int counter = 0;
```

```
pthread_mutex_t mutex;
```

```
void *increment(void *arg)
```

```
{  
    for (int i = 0; i < 100000; i++)  
    {  
        pthread_mutex_lock(&mutex);  
  
        counter++;  
  
        pthread_mutex_unlock(&mutex);  
    }  
}
```

```
    return NULL;  
}
```

```
int main()
```

```
{  
    pthread_t t1, t2;  
  
    pthread_mutex_init(&mutex, NULL);
```

```

pthread_create(&t1, NULL, increment, NULL);
pthread_create(&t2, NULL, increment, NULL);

pthread_join(t1, NULL);
pthread_join(t2, NULL);

printf("Final Counter Value: %d\n", counter);

pthread_mutex_destroy(&mutex);

return 0;
}

```

Compilation

gcc mutex.c -o mutex -pthread

Execution

./mutex

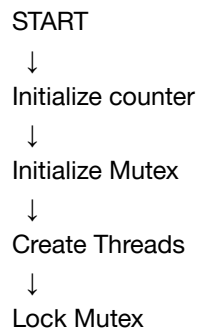
Output

Final Counter Value: 200000

Comparison

Feature	Race Condition	Mutex
Synchronization	No	Yes
Final value	May be incorrect	Correct
Data safety	Unsafe	Safe
Execution	May be slightly faster	Slight overhead
Shared access	Concurrent	One thread at a time

Flow Chart



↓
Increment Counter
↓
Unlock Mutex
↓
More iterations?
↓
pthread_join()
↓
Display Counter
↓
Destroy Mutex
↓
STOP

Observation

Without a mutex, multiple threads can update the counter simultaneously, causing lost updates. With a mutex, only one thread accesses the critical section at a time, so the final value is correct.

Output:

```
simra@simran315:~$ nano mutex.c
simra@simran315:~$ gcc mutex.c -o mutex -pthread
simra@simran315:~$ ./mutex
Final Counter Value: 200000
```

Result

The program was successfully modified using mutex synchronization. The mutex eliminated the race condition and produced the expected counter value, with a small synchronization overhead.